

Responsive Web Design

ผศ. ดร. เกี๊จแกว ธเนศวร

kejkaew.tha@kmutt.ac.th

การบ้าน

- สร้าง Portfolio web page ของนักศึกษาแต่ละคนสำหรับใช้เพื่อการสมัครงาน โดยใช้ HTML ง่าย ๆ และมีรายละเอียดข้อมูลดังนี้
- ทำการตกแต่ง Portfolio webpage ของนักศึกษา โดยใช้ CSS แบบ External CSS นะ
- ทำการตกแต่ง layout ของนักศึกษา เพิ่มเติม

๒ การบ้าน

- ทำ html पेจอีกหนึ่งเพจมีชื่อว่า project.html
- เนื้อหาของเพจเป็นเรื่องเกี่ยวกับ project ต่าง ๆ ที่เราเคยทำหรือคิดว่าจะทำ มีมากกว่า 1 อย่างก็ได้
 - เช่น project จากที่เคยทำในห้องเรียน
- ใช้ CSS file รวมกับ portfolio html file ของเรา

ตารางเรียน

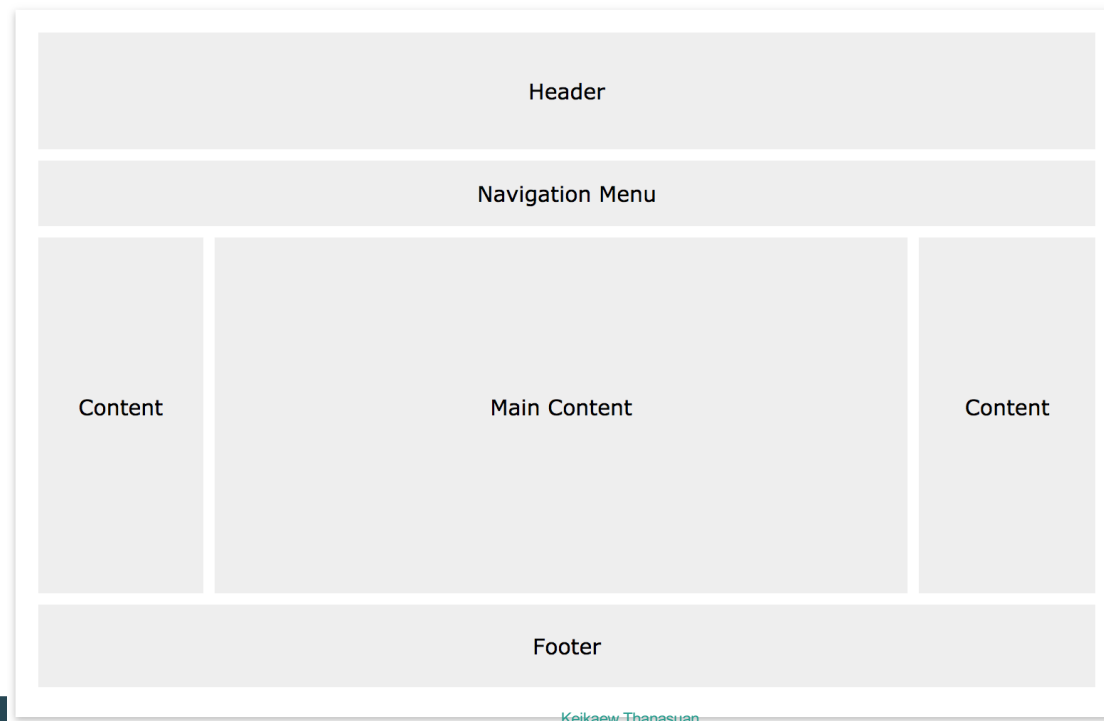
สัปดาห์	วันที่	หัวข้อ/รายละเอียด	Assignment (ผู้ตรวจ)	ส่ง Assignment
1	4/8/2569	Introduction to Web Development & HTML5	Assignment 1 (KT,KN)	1
2	11/8/2569	CSS3 Fundamentals	Assignment 2 (KT, KN)	1,2
3	18/8/2569	Advanced CSS & UI Layouts	Assignment 3 (KT)	1 2 3
4	25/8/2569	Responsive Web Design	Assignment 4 (KT, KN)	2,3,4
5	1/9/2569	JavaScript 1: Variables, Data Types & Logic Control JavaScript 1 + DOM	Assignment 5 (KT, KN)	3,4,5
6	8/9/2569	JavaScript 2: Arrays, Objects & Logic Drills	Assignment 6 (KT)	4,5,6
7	15/9/2569	JavaScript 3: DOM Manipulation & Event Handling	Assignment 7 (KT, KN)	5,6,7
8	22/9/2569	Asynchronous JS, AJAX & JSON Data Flow	Assignment 8 (KT, KN)	6,7,8
9	29/9/2569	Introduction to Node.js & Express Basics	Assignment 9 (KT)	7,8,9
10	6/10/2569	Backend Architecture & RESTful API (CRUD)	Assignment 10 (KT, KN)	8,9,10
	13/10/2569	หยุด		
	20/10/2569	หยุด		
	27/10/2569	หยุด		
11	3/11/2569	Microservices Architecture Concept & Hands-on	Assignment 11 (KT, KN)	9,10,11
12	10/11/2569	Database Integration (MySQL / Firebase)	Assignment 12 (KT, KN)	10,11,12
13	17/11/2569			11,12
14	24/11/2569	ส่งโปรเจค (9:00-16:00)	KT	12
15	12/2569	สอบปลายภาค ในตาราง		

วันนี้เราจะเรียนเรื่อง...

- CSS Website layout
- Responsive Web Design
- Assignment ในห้องเรียน
- Quiz
 - เปิดเนื้อหา สไลด์ สรุป code ต่างๆ ได้หมด
 - ห้ามใช้ AI ทุกประเภท ห้ามถามเพื่อน ห้ามแชท ห้ามคุย
 - สอบในรอบที่เรียน
 - 11.00 - 11:30 น..
 - 15:30 – 16.00 น.
 - อย่าลืม ดินสอ ปากกา

Website Layout

- โดยปกติแล้ว website จะแบ่งออกเป็น headers, menus, content และ footer:



Header

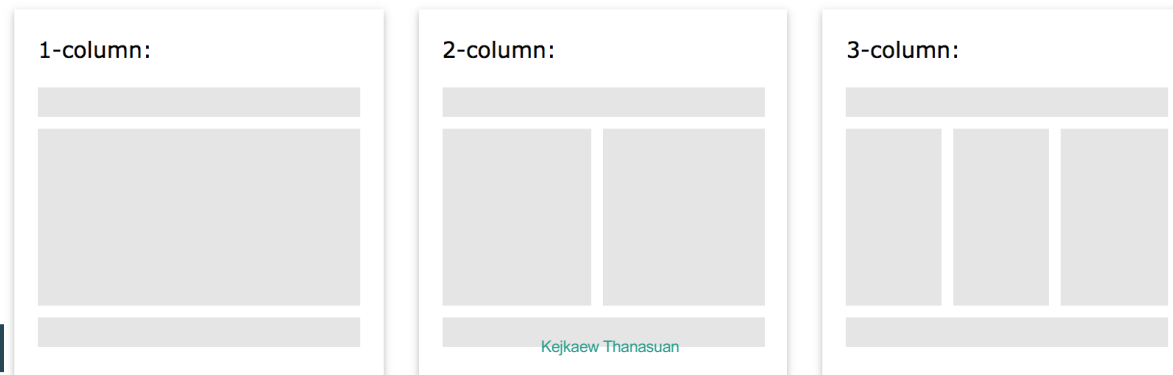
- A header is usually located at the top of the website (or right below a top navigation menu). It often contains a logo or the website name:



Content

การจัดเลย์เอาต์ในส่วนนี้ มักขึ้นอยู่กับกลุ่มผู้ใช้งานเป้าหมาย โดยเลย์เอาต์ที่พบได้บ่อยที่สุดจะเป็นแบบใดแบบหนึ่ง (หรือนำมารวมกัน) ดังต่อไปนี้:

- **แบบ 1 คอลัมน์** (มักใช้สำหรับเบราร์วเซอร์บนมือถือ)
- **แบบ 2 คอลัมน์** (มักใช้สำหรับแท็บเล็ตและแล็ปท็อป)
- **แบบ 3 คอลัมน์** (ใช้สำหรับคอมพิวเตอร์เดสก์ท็อปเท่านั้น)



Footer

- ส่วนท้าย (Footer) จะถูกวางไว้ที่ด้านล่างสุดของหน้าเว็บ โดยมักประกอบด้วยข้อมูล เช่น ข้อมูลลิขสิทธิ์ และข้อมูลการติดต่อ:

REPORT ERROR	PRINT PAGE	FORUM	ABOUT
Top 10 Tutorials HTML Tutorial CSS Tutorial JavaScript Tutorial W3.CSS Tutorial Bootstrap Tutorial SQL Tutorial PHP Tutorial jQuery Tutorial Angular Tutorial How To Tutorial	Top 10 References HTML Reference CSS Reference JavaScript Reference W3.CSS Reference Bootstrap Reference Browser Statistics PHP Reference HTML Colors jQuery Reference AngularJS Reference	Top 10 Examples HTML Examples CSS Examples JavaScript Examples W3.CSS Examples Bootstrap Examples HTML DOM Examples PHP Examples jQuery Examples Angular Examples XML Examples	Web Certificates HTML Certificate CSS Certificate JavaScript Certificate jQuery Certificate PHP Certificate Bootstrap Certificate XML Certificate

W3Schools is optimized for learning, testing, and training. Examples might be simplified to improve reading and basic understanding. Tutorials, references, and examples are constantly reviewed to avoid errors, but we cannot warrant full correctness of all content. While using this site, you agree to have read and accepted our terms of use, cookie and privacy policy. Copyright 1999-2018 by Refsnes Data. All Rights Reserved.

Powered by W3.CSS.

Responsive Web Design

- ทำไมเราถึงต้องมีการทำ Responsive Web Design?
- Responsive web design จะทำให้ website ของเรา ดูดีในทุกอุปกรณ์แสดงผล (โทรศัพท์ คอมพิวเตอร์ หรือ Tablet)
- Responsive web design จะใช้แค่ HTML and CSS ไม่มี JavaScript



Desktop



Tablet



Phone

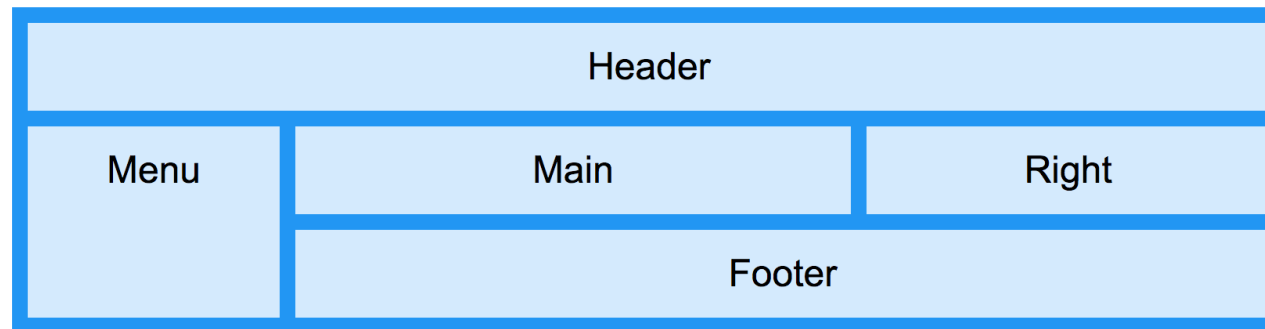
Setting The Viewport

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

- ใน header ของ HTML file

CSS Grid Layout Module

- The CSS Grid Layout Module คือ การสร้าง rows และ columns บน web page ทำให้ง่ายในการ design web page โดยเราไม่จำเป็นต้องใช้ floats และ position เข้ามาช่วย
- สามารถทำได้โดยตั้งค่า display เป็น grid หรือ inline grid ใน container ใหญ่ และ element ที่อยู่ใน container นั้นจะกลายเป็น grid items



CSS Grid Container

- **grid-template-columns** : คุณสมบัตินี้ใช้กำหนดจำนวนคอลัมน์ในเลย์เอาต์แบบกริด และสามารถกำหนดความกว้างของแต่ละคอลัมน์ได้ ตัวอย่างเช่น

```
.grid-container {  
    display: grid;  
    grid-template-columns: auto auto auto auto;  
}
```

ตัวอย่างเช่นใน demo-slide.html

HTML

```
12 <!-- Basic Grid Container แบบไม่มี Content ตามสไลด์ -->
13 <div class="grid-container">
14   <div class="grid-card">1 (Header)</div>
15   <div class="grid-card">2 (Menu)</div>
16   <div class="grid-card">3 (Main)</div>
17   <div class="grid-card">4 (Right)</div>
18   <div class="grid-card">5 (Footer)</div>
19 </div>
20
```

CSS

```
22 .grid-container {
23   display: grid;
24   grid-template-columns: 14% auto 22%;
25   gap: 18px;
26   max-width: 1200px;
27   margin: 0 auto;
28 }
```



The **grid-template-rows** Property

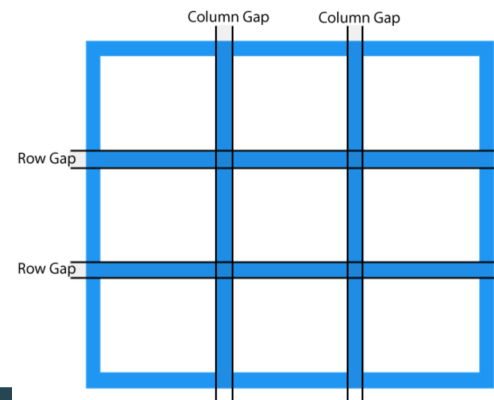
- คุณสมบัติ `grid-template-rows` ใช้กำหนดความสูงของแต่ละแถว (ในเลย์เอาต์แบบ CSS Grid) (กำหนดความสูงของแต่ละแถว).

- ตัวอย่างเช่น

```
.grid-container {  
    display: grid;  
    grid-template-rows: 80px 200px;  
}
```

Grid Gaps

- พื้นที่ว่างระหว่างแต่ละคอลัมน์หรือแถว เรียกว่า gaps
- `grid-column-gap` : กำหนดระยะห่างระหว่างคอลัมน์
- `grid-row-gap` : กำหนดระยะห่างระหว่างแถว
- `grid-gap` : กำหนดทั้งระยะห่างระหว่างคอลัมน์และระยะห่างระหว่างแถวพร้อมกัน
- `gap` : เหมือนกับ `grid-gap`



ตัวอย่าง

```
22 .grid-container {  
23     display: grid;  
24     grid-template-columns: 14% auto 22%;  
25     gap: 18px;  
26     max-width: 1200px;  
27     margin: 0 auto;  
28 }
```

CSS



CSS Grid Item (div ที่อยู่ใน Grid container)

- The grid-column Property: กำหนดว่า item ของเราจะเริ่มจาก column ที่เท่าไร ไปถึง column ที่เท่าไร
- The grid-row Property: กำหนดว่า item ของเราจะเริ่มจาก row ที่เท่าไร ไปถึง row ที่เท่าไร
- The grid-area Property : คือการนำ grid-column Property และ grid-row Property มารวมไว้ด้วยกัน

ตัวอย่าง

เริ่มที่

ถึง **column**
ที่ 2

```
#item1{  
  grid-column: 1 / 3;  
}
```

เริ่มที่

กว้าง

```
#item1{  
  grid-row: 1 / span 3;  
}
```

```
#item1{  
  grid-column: 1 / span 3;  
}
```

row

row

```
#item1{  
  grid-area: 1 / 1 / 3 / 3;  
}
```

column

column

ตัวอย่าง



```
#header{  
  grid-column: 1 / span 3;  
}  
  
#menu{  
  grid-column: 1 / span 1;  
}  
  
#main{  
  grid-column: 2 / span 1;  
}  
  
#right{  
  grid-column: 3 / span 1;  
}  
  
#footer{  
  grid-column: 1 / span 3;  
}
```

นอกจากนี้

- สำหรับ `grid-template-areas` สามารถตั้งชื่อ Layout เช่น "header header" "sidebar main" "footer footer" แยกในแต่ละแถว จะช่วยให้มองภาพรวมการวางหน้าเว็บได้ง่ายกว่าการนับตัวเลข line number / span
- ตัวอย่างเช่น

1 column

```
grid-template-areas:  
  "header"  
  "menu"  
  "main"  
  "sidebar"  
  "footer";
```

2 columns

```
grid-template-areas:  
  "header  header"  
  "menu    menu"  
  "main    sidebar"  
  "footer  footer";
```

3 columns

```
grid-template-areas:  
  "header  header  header"  
  "menu    main    sidebar"  
  "footer  footer  footer";
```

CSS: Media Query

- Media query is a CSS technique introduced in CSS3.
- โดย CSS properties ที่อยู่ใน media จะทำงานก็ต่อเมื่อ เงื่อนไขใน @media rule เป็นจริง เช่น

rule

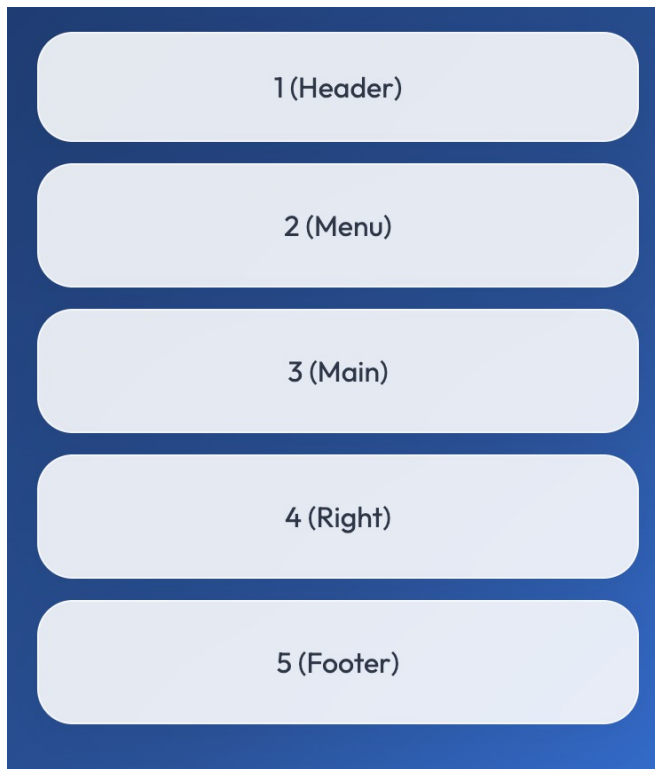
- @media only screen and (max-width: 500px) {
 body {
 background-color: lightblue;
 }

ตัวอย่าง 2 columns



```
@media only screen and (max-width: 767px){  
  .grid-container{  
    display: grid;  
    grid-template-columns: auto 20%;  
    grid-gap: 10px;  
  }  
  #header{  
    grid-column: 1 / span 2;  
  }  
  #menu{  
    grid-column: 1 / span 2;  
  }  
  #main{  
    grid-column: 1 / span 1;  
  }  
  #right{  
    grid-column: 2 / span 1;  
  }  
  #footer{  
    grid-column: 1 / span 2;  
  }  
}
```

ตัวอย่างสำหรับ mobile (1 column)



```
@media only screen and (max-width: 500px){  
  .grid-container{  
    display: grid;  
    grid-template-columns: auto;  
    grid-gap: 10px;  
  }  
  #header{  
    grid-column: 1 / span 1;  
  }  
  
  #menu{  
    grid-column: 1 / span 1;  
  }  
  
  #main{  
    grid-column: 1 / span 1;  
  }  
  #right{  
    grid-column: 1 / span 1;  
  }  
  #footer{  
    grid-column: 1 / span 1;  
  }  
}
```

Pokémon web design

1



2

Navigation

⚡ Profile

📊 Base
Stats

🐾 Friends

📈 CP
Values

📰 News

3

Hello, **Pikachu**

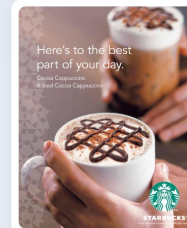


- **Weight:** 6 kg.
- **Height:** 0.4 m.
- **Max CP** = 938

Stat	Value
ATK	112
DEF	96

4

Newsletter



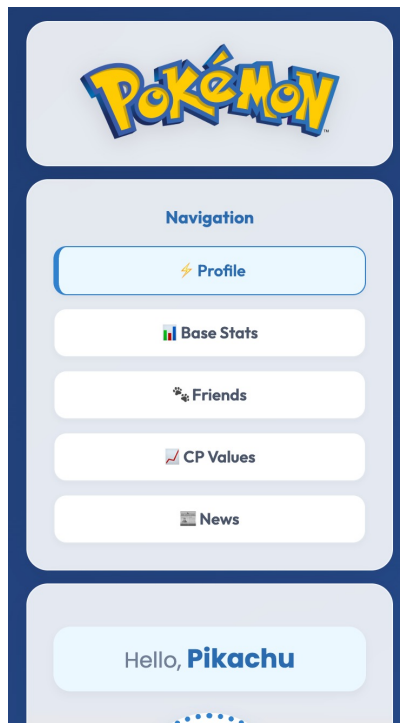
5

If you want to know more, please click!:

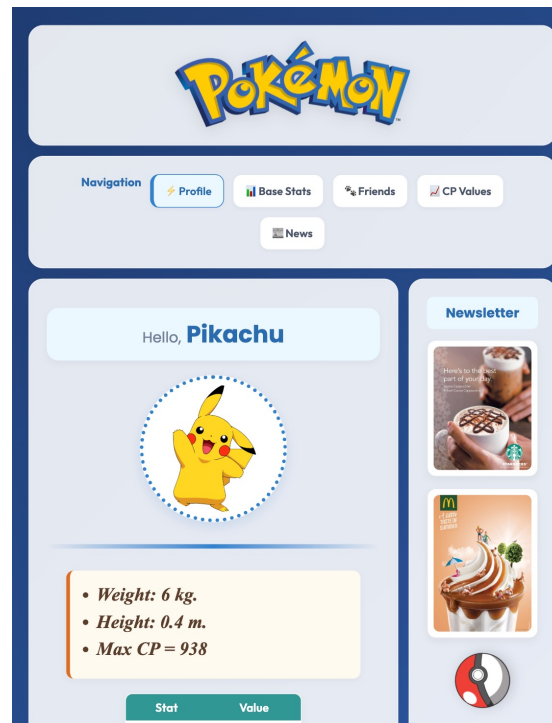
[Reference](#)

Images from:

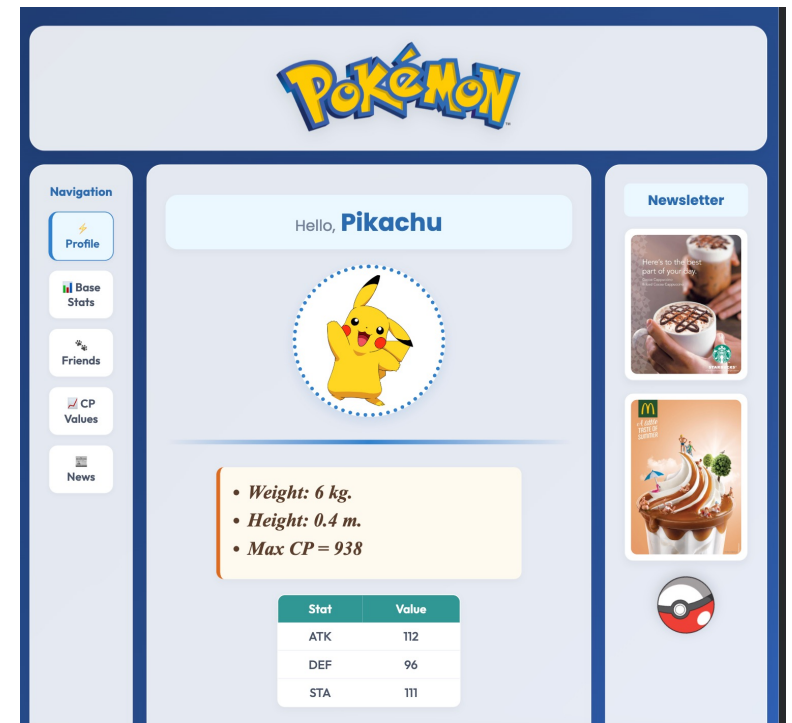
[Reference](#)



1 column



2 columns



3 columns

Modern Front-end CSS

- การใช้ @media (max-width: ...) เป็นการออกแบบแบบ Desktop-First (เขียนจอใหญ่ก่อนแล้วค่อยบีบลงมือถือ)
- **สิ่งที่ควรทำ : ควรออกแบบแบบ Mobile-First Approach** โดยใช้ @media (min-width: ...) เพราะเป็นมาตรฐานที่การเขียน web และ CSS Framework สมัยใหม่ (เช่น Tailwind, Bootstrap) นิยมใช้มากกว่า
- ดูตัวอย่างได้ใน modern.css

Modern Grid Techniques (modern.css)

- **Fraction Unit (fr)** : ใน main.css ส่วนใหญ่เน้นใช้ % และ auto แต่เราสามารถ
ใช้หน่วย 1fr 2fr 1fr ซึ่งเป็นหัวใจหลักของ CSS Grid ยุคใหม่ เพราะคำนวณพื้นที่
เหลือให้อัตโนมัติและไม่ติดปัญหาเรื่อง padding/gap ล้นจอ
- **Responsive Grid โดยไม่ใช้ Media Query (ใช้ repeat(), auto-fit, minmax())** : เช่น

```
grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
```

Modern Grid Techniques (modern.css)

- Responsive Image & Media (การใช้ `max-width: 100%; height: auto;`)
: ปัญหาคลาสสิกคือรูปภาพล้นการ์ดหรือสัดส่วนเบี้ยวเมื่อย่อจอ
- เราสามารถแก้ไขได้ เช่น

```
.logo-img {  
  width: 320px;  
  max-width: 90%;  
  height: auto;  
  filter: drop-shadow(0 4px 10px rgba(0, 0, 0, 0.1));  
}
```

Modern Grid Techniques (modern.css)

- หน่วย rem vs px : ทำไมการทำ Responsive ถึงควรเริ่มใช้ rem แทน px เพื่อรองรับ Accessibility เมื่อผู้ใช้ปรับขนาดตัวอักษรในเครื่อง
 - rem คือ หน่วยวัดความยาวสัมพัทธ์ (Relative Unit) ที่อ้างอิงขนาดมาจาก ขนาดฟอนต์ของเว็บ คือ $16\text{px} = 1\text{rem}$
 - ปัญหาของ px (Hardcoded): ถ้าเราใส่ font-size: 16px; เมื่อผู้สูงอายุไปปรับขนาดตัวอักษรใน Browser Setting ให้เป็น "Large / ขนาดใหญ่" ตัวหนังสือในเว็บเราจะไม่ยอมขยายตาม เพราะโดน px ล็อกไว้
 - แต่ถ้าเราใส่ font-size: 1rem; เมื่อผู้ใช้ปรับขนาดฟอนต์ในเครื่อง หน้าเว็บของเราจะ ขยายใหญ่อ่านง่ายตามความต้องการของผู้ใช้ทันที

CSS Specificity

ถ้า element เดียวกัน แต่มีการระบุ CSS rule มากกว่า 1 แบบ จะต้องใช้ CSS ตัวใด

- **Inline styles** - Example: `<h1 style="color: #ffffff;">`.
- **IDs** - as `#navbar`.
- **Classes, attributes and pseudo-classes** - This category includes `.classes`, `[attributes]` and pseudo-classes such as `:hover`, `:focus` etc.
- **Elements and pseudo-elements** - This category includes element names and pseudo-elements, such as `h1`, `div`, `:before` and `:after`.

How to Calculate Specificity?

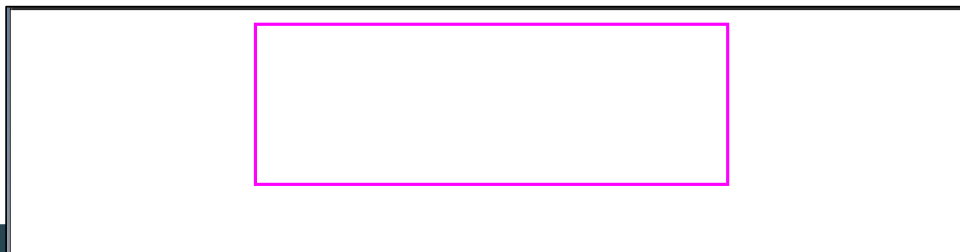
เริ่มคำนวณจาก 0

- + 1000 for style attribute
- + 100 for each ID
- + 10 for each attribute, class or pseudo-class
- + 1 for each element name or pseudo-element.
- สุดท้าย CSS rule แบบไหน มีค่ามากกว่า เลือกอันนั้น

ตัวอย่าง

```
<div>  
  <p class="class1"></p>  
</div>  
<hr>
```

ผลที่ได้



```
div .class1{  
  border: 2px solid magenta;  
  margin: auto;  
  width: 50%;  
  height: 100px;  
}  
p{  
  border: 2px solid rgb(6, 58, 23);  
  margin: auto;  
  width: 30%;  
  height: 100px;  
}  
@media only screen and (max-width: 767px){  
  .class1{  
    border: 2px solid blue;  
  }  
}
```

Assignment 4

- จาก Pizza dip ให้ตกแต่งเว็บเพจ โดยใช้
 - Grid display
 - พยายามทำ webpage ให้เป็น Responsive โดยใช้ Media Queries
 - พยายามกำหนด width เป็นแบบ percent
 - เปลี่ยนสี, padding, size etc.
- ลอง design layout สำหรับ grid display ก่อน โดยการทำ responsive ทั้งหมดก่อน

เริ่มสอบ 11:00
สอบ Quiz 20 นาที